

Implementación del Algoritmo ACO

Resolución de Laberintos mediante Optimización por Colonias de Hormigas

Juan Correa
Juan Merchan
Samuel Lagos Prado

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Marco Teórico	3
3.1. Algoritmo ACO	3
3.2. Feromonas	4
4. Probabilidad de Transición	4
5. Función Heurística	4
6. Actualización de Feromonas	4
7. Generación del Laberinto	5
8. Explicación del Código	5
8.1. Constantes	5
8.2. Construcción de Soluciones	5
8.3. Selección Probabilística	6
8.4. Evaporación	6
9. Visualización	6
10. Parámetros Utilizados	6
11. Análisis de Resultados	6
12. Complejidad Computacional	7
13. Ventajas del Algoritmo	7

14.Desventajas	7
15.Aplicaciones Reales	8
16.Conclusiones	8
17.Referencias	8

1. Introducción

La Optimización por Colonias de Hormigas (ACO, *Ant Colony Optimization*) es un algoritmo bioinspirado basado en el comportamiento de las hormigas reales cuando buscan caminos entre su colonia y una fuente de alimento.

Las hormigas depositan feromonas sobre el suelo mientras se desplazan. Los caminos con mayor concentración de feromonas tienden a atraer más hormigas, lo que provoca que las rutas más cortas y eficientes se refuercen progresivamente.

En este proyecto se implementó un sistema capaz de:

- Generar un laberinto aleatorio.
- Simular múltiples hormigas explorando el entorno.
- Aplicar evaporación y depósito de feromonas.
- Encontrar rutas eficientes entre un origen y una meta.
- Visualizar gráficamente la evolución del algoritmo.

2. Objetivos

2.1. Objetivo General

Implementar y visualizar el algoritmo ACO para resolver laberintos generados aleatoriamente utilizando Python.

2.2. Objetivos Específicos

- Comprender el funcionamiento del algoritmo ACO.
- Generar laberintos mediante backtracking recursivo.
- Implementar reglas probabilísticas de transición.
- Simular el depósito y evaporación de feromonas.
- Analizar cómo el sistema converge hacia rutas óptimas.

3. Marco Teórico

3.1. Algoritmo ACO

El algoritmo ACO pertenece a la categoría de algoritmos de inteligencia de enjambre (*Swarm Intelligence*).

Su funcionamiento se basa en:

1. Varias hormigas exploran el entorno.
2. Cada hormiga toma decisiones probabilísticas.

3. Las mejores rutas reciben más feromonas.
4. Las feromonas débiles se evaporan.
5. El sistema converge gradualmente hacia rutas óptimas.

3.2. Feromonas

Las feromonas representan memoria colectiva.

Si una ruta es buena:

$$\tau_{ij} \uparrow$$

Si una ruta es mala:

$$\tau_{ij} \downarrow$$

donde:

$$\tau_{ij}$$

representa la cantidad de feromona entre los nodos i y j .

4. Probabilidad de Transición

La probabilidad de que una hormiga se mueva desde el nodo i hacia el nodo j se define como:

:contentReference[oaicite:0]index=0

donde:

- τ_{ij} : feromona.
- η_{ij} : heurística.
- α : importancia de feromonas.
- β : importancia de la heurística.

5. Función Heurística

La heurística utilizada es la distancia Manhattan inversa:

:contentReference[oaicite:1]index=1

Esta fórmula favorece nodos cercanos a la meta.

6. Actualización de Feromonas

Las feromonas se evaporan después de cada iteración:

:contentReference[oaicite:2]index=2

donde:

- ρ : tasa de evaporación.

Luego, las hormigas depositan nuevas feromonas:
:contentReference[oaicite:3]index=3
donde:

- Q : cantidad de feromona.
- L : longitud del camino.

7. Generación del Laberinto

El laberinto se genera mediante un algoritmo de backtracking recursivo.
El proceso consiste en:

1. Inicializar todo como paredes.
2. Seleccionar una celda aleatoria.
3. Excavar caminos recursivamente.
4. Romper algunas paredes aleatorias para aumentar complejidad.

El origen se ubica en:

$$(1, 0)$$

La meta se ubica en:

$$(alto - 2, ancho - 1)$$

8. Explicación del Código

8.1. Constantes

```
1 PARED    = 0
2 CAMINO   = 1
3 ORIGEN   = 2
4 META     = 3
```

Representan cada tipo de celda del laberinto.

8.2. Construcción de Soluciones

Cada hormiga:

- Parte desde el origen.
- Evalúa vecinos disponibles.
- Calcula probabilidades.
- Escoge un siguiente nodo.
- Repite hasta llegar a la meta.

8.3. Selección Probabilística

Las hormigas no siempre eligen el mejor camino directamente. Esto evita que el algoritmo quede atrapado en soluciones locales.

8.4. Evaporación

La evaporación permite eliminar información obsoleta y mantener exploración. Sin evaporación:

- El algoritmo convergería demasiado rápido.
- Se perdería diversidad.

9. Visualización

La simulación utiliza Matplotlib para mostrar:

- Paredes en color negro.
- Origen en verde.
- Meta en rojo.
- Feromonas en tonos naranjas.
- Mejor camino en amarillo.

La animación permite observar cómo las feromonas se concentran gradualmente en la mejor ruta.

10. Parámetros Utilizados

Parámetro	Valor
Hormigas	50
Iteraciones	60
Alpha	1
Beta	5
Evaporación	0.25
Q	100

Cuadro 1: Parámetros del algoritmo

11. Análisis de Resultados

Se observó que:

- Inicialmente las hormigas exploran rutas aleatorias.

- Las rutas cortas acumulan más feromonas.
- La colonia converge hacia un camino eficiente.
- El algoritmo mejora progresivamente la solución.

También se comprobó que:

- Un valor alto de beta favorece la heurística.
- Una evaporación excesiva dificulta la convergencia.
- Muy pocas hormigas reducen la exploración.

12. Complejidad Computacional

La complejidad aproximada es:

:contentReference[oaicite:4]index=4

Esto depende directamente de:

- Tamaño del laberinto.
- Cantidad de hormigas.
- Número de iteraciones.

13. Ventajas del Algoritmo

- Excelente para optimización.
- Adaptable a problemas complejos.
- Capacidad de aprendizaje colectivo.
- Buena exploración del espacio de búsqueda.

14. Desventajas

- Puede requerir muchas iteraciones.
- Sensible a parámetros.
- Mayor costo computacional en grandes entornos.

15. Aplicaciones Reales

El algoritmo ACO se utiliza en:

- Optimización de rutas.
- Redes de telecomunicaciones.
- Logística y transporte.
- Inteligencia artificial.
- Robótica.
- Planificación de tráfico.

16. Conclusiones

- El algoritmo ACO permite resolver laberintos de forma eficiente.
- La inteligencia colectiva mejora progresivamente las soluciones.
- Las feromonas funcionan como memoria compartida.
- La visualización facilita comprender el comportamiento emergente.

El proyecto demuestra cómo algoritmos inspirados en la naturaleza pueden aplicarse a problemas complejos de optimización.

17. Referencias

- Dorigo, Marco. Ant Colony Optimization.
- Russell & Norvig. Artificial Intelligence: A Modern Approach.
- Documentación oficial de Python.
- Documentación de Matplotlib.